

No Clicks allowed - Let's setup Vaultwarden!

Hey! My name is Noe and the reason why I'm going to talk to you for the next 5 - 15" is Vaultwarden.

More precisely I got annoyed with Liquid Glass adds and random notifications urging me to use AI.

With the recent C39C announcement of initial usb-c display support under Asahi Linux I began a new iteration of systems and software I use.

Terms

- Tailscale
 - Terraform
 - Nix
 - VIP
-

Getting a hostname

A few weeks before I started this side quest I started investigating HTTPS certificates for my tailnet.

monitoring.yak-bebop.ts.net

machine name

tailnet name

Tailscale Services

Instead of connecting to specific devices, users connect to Tailscale Services using stable MagicDNS names while Tailscale automatically routes traffic to available hosts across your infrastructure.

Previously on fun with Tailscale

Around end of last year I got interested in a VM based setup to test system deployment via `nixos-anywhere`.

I remember begin to lazy to lookup how to change my libvirt setup to a bridge interface so I decided to bake a use-once Tailscale auth key into my VM.

This was the first time I used the Tailscale Terraform provider to generate these auth keys.

So when I read "[...] you need to define a Service through the admin console [...]" I immediately checked the Terraform provider to see if I can use this instead.

No support lets check GitHub

1. [Add Tailscale Services to the provider](#)
2. [Add support for Services endpoints](#)

Unfortunately there was no support yet. But I wasn't the only one looking for a no click solution to this.

Compare the code

To arrive at my version of this I searched through <https://github.com/tailscale/tailscale> the official repository and found the `VIPServices` code path

Which I then adapted with inspiration of the existing `users.go` code.

Not only did this solve the deprecation warning it also made the code way cleaner

The end result is matching the merged code 1/1. Which means I could have contributed my changes all along!

Fun fact I'm pretty sure the contributor took the same route:

1. They are using `vip-services` as endpoint

Actually improvements of my version. My version correctly uses `tcp` in front of port number.

```
jj diff --to xvm
```

`body[T](res, req)`

```
func body[T any](resource doer, req *http.Request) (*T, error) {  
    t, _, err := bodyWithResponseHeader[T](resource, req)  
    return t, err  
}
```

```
func bodyWithResponseHeader[T any](resource doer, req  
*http.Request) (*T, http.Header, error) {  
    var v T  
    header, err := resource.doWithResponseHeaders(req, &v)  
    if err != nil {  
        return nil, nil, err  
    }  
    return &v, header, nil  
}
```

I never bothered to figure how exactly this is implemented. But on first glance it looked not very Goish to me.

Embedded client

```
type Client struct {  
    ...  
    // Specific resources  
    vipServices *VIPServicesResource  
    ...  
}
```

```
type VIPServicesResource struct {  
    *Client  
}
```

- Is this a common pattern in Go?
 - What is the motivation for this?
-

Using the client

Now that the client is implemented we can start working on the provider.

Luckily the "use your local package" story of Go is straight forward. Next thing is running the tests.

In my first GoTalk I heard Lukas saying:

"The first thing I do when I checkout Go projects is running tests."

This really resonated with me and I got the say the integrated testing story together with the fast compile times are definitely something that will draw me back to Go.

Integration Testing

Adding one of my existing hosts to a different tailnet is only usable when working from home.

Instead lets setup a vm. My tool of choice for such experiments is `microvm.nix` and again use the terraform provider to generate a new auth key.

Update via data source

```
data "tailscale_vip_service" "created" {
  name = "svc:example"
}

resource "tailscale_vip_service" "test_service" {
  name      = "svc:example-updated"
  comment   = "Updated service"
  ports     = ["tcp:443", "tcp:80"]
  addrs     = data.tailscale_vip_service.created.addrs
}
```

since the address are computed **and** we must supply them on update we could try to get them via an data source

But this also requires us to update the client instead of just using the name we need to decouple the current of the updated name

So I implemented a new interface in the client and that's why I didn't push my changes.

How would you approach this?

ForceNew

In the end a simple `force_new` would have been a way simpler solution!

Testing with local providers

Configuring a new service